

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.
import kagglehub
manishkr1754_cardekho_used_car_data_path = kagglehub.dataset_download('manishkr1754/cardekho-used-car-data')

print('Data source import complete.')
```

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a new session
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/manishkr1754/cardekho-used-car-data/cardekho_dataset.csv
```

```
import os

for item in os.listdir("/kaggle/input"):
    print(item)
```

```
datasets
```

1. Load the Dataset

The dataset is provided by Kaggle as a CSV file.

We will use Pandas to read the CSV file into a DataFrame.

A DataFrame is a two-dimensional table containing rows and columns. It allows us to inspect, clean, analyze, and prepare the dataset for machine learning.

```
import pandas as pd

dataset_path = "/kaggle/input/datasets/manishkr1754/cardekho-used-car-data/cardekho_dataset.csv"

df = pd.read_csv(dataset_path)

print("Dataset loaded successfully.")
```

```
Dataset loaded successfully.
```

```
print(df.shape)
```

```
(15411, 14)
```

2. Understand the Dataset

Before training a machine learning model, we need to understand what information is available in the dataset.

We will inspect:

- column names
- data types
- sample records
- missing values
- duplicate records
- numerical statistics

This process is called Exploratory Data Analysis (EDA).

```
print(df.columns)
```

```
Index(['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',
      'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',
      'max_power', 'seats', 'selling_price'],
      dtype='object')
```

```
df.head()
```

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	Petrol	Manual	18.90
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	Petrol	Manual	17.00

3. Inspect Data Types

Different columns contain different types of information.

Numerical columns can usually be used directly by machine-learning algorithms, while categorical/text columns need to be converted into numerical representations before training.

We will inspect the data types before deciding how to preprocess the data.

```
df.dtypes
```

```
Unnamed: 0      int64
car_name        object
brand           object
model           object
vehicle_age     int64
km_driven       int64
seller_type     object
fuel_type       object
transmission_type object
mileage         float64
engine          int64
max_power       float64
seats           int64
selling_price   int64
dtype: object
```

4. Check Missing Values

Machine-learning models generally cannot work directly with missing values in the way raw datasets contain them.

We will identify which columns contain missing values before deciding how to handle them.

```
df.isnull().sum()
```

```
Unnamed: 0      0
car_name        0
brand           0
model           0
vehicle_age     0
km_driven       0
seller_type     0
fuel_type       0
transmission_type 0
mileage         0
engine          0
max_power       0
```

```
seats      0
selling_price 0
dtype: int64
```

5. Check Duplicate Records

Duplicate rows can cause the same car record to appear more than once.

We will count duplicate rows before training the model.

```
df.duplicated().sum()
```

```
np.int64(0)
```

6. Numerical Statistics

We will summarize the numerical columns to understand their typical values and ranges.

This helps us detect unusual values and understand the scale of the data before machine learning.

```
df.describe()
```

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats	selling_price
count	15411.000000	15411.000000	1.541100e+04	15411.000000	15411.000000	15411.000000	15411.000000	1.541100e+04
mean	9811.857699	6.036338	5.561648e+04	19.701151	1486.057751	100.588254	5.325482	7.749711e+05
std	5643.418542	3.013291	5.161855e+04	4.171265	521.106696	42.972979	0.807628	8.941284e+05
min	0.000000	0.000000	1.000000e+02	4.000000	793.000000	38.400000	0.000000	4.000000e+04
25%	4906.500000	4.000000	3.000000e+04	17.000000	1197.000000	74.000000	5.000000	3.850000e+05
50%	9872.000000	6.000000	5.000000e+04	19.670000	1248.000000	88.500000	5.000000	5.560000e+05
75%	14668.500000	8.000000	7.000000e+04	22.700000	1582.000000	117.300000	5.000000	8.250000e+05
max	19543.000000	29.000000	3.800000e+06	33.540000	6592.000000	626.000000	9.000000	3.950000e+07

7. Understand Categorical Feature Cardinality

Not every categorical column should necessarily be encoded in exactly the same way.

Before choosing an encoding strategy, we need to understand how many different categories each categorical feature contains.

For example, `fuel_type` usually has relatively few categories, while `model` may contain many different categories.

We will inspect the categorical features using:

- `value_counts()` to see how frequently each category appears.
- `nunique()` to count how many different categories exist.

What does `nunique()` mean?

`nunique()` means **number of unique values**.

For example, if a column contains:

Petrol

Petrol

Diesel

Petrol

CNG

there are 3 unique values:

- Petrol
- Diesel
- CNG

Therefore:

```
nunique()
```

```
df["fuel_type"].value_counts()
```

```
fuel_type
Petrol      7643
Diesel      7419
CNG         301
LPG         44
Electric     4
Name: count, dtype: int64
```

```
df["brand"].nunique()
```

```
32
```

```
df["model"].nunique()
```

```
120
```

```
df["fuel_type"].nunique()
```

```
5
```

8. Inspect Categorical Feature Distributions

We have identified that categorical features can contain different numbers of categories.

Before choosing an encoding strategy, we will inspect the distribution of each categorical feature.

`value_counts()` shows how frequently each category occurs.

This is useful because a category that appears only a few times may behave differently from a category with thousands of examples.

We will inspect:

- brand
- model
- seller_type
- fuel_type
- transmission_type

```
categorical_columns = [
    "brand",
    "model",
    "seller_type",
    "fuel_type",
    "transmission_type"
]

for column in categorical_columns:
    print(f"\n===== {column} =====")
    print(df[column].value_counts())
```

```
===== brand =====
brand
Maruti      4992
Hyundai     2982
Honda       1485
Mahindra    1011
Toyota      793
Ford        790
Volkswagen  620
Renault     536
BMW         439
Tata        430
Mercedes-Benz 337
Skoda       334
Audi        192
Datsun      170
Jaguar       59
Land Rover  51
Jeep        41
Kia         32
Porsche     21
Volvo       20
MG          19
Mini        17
Nissan      11
Lexus       10
Isuzu       8
Bentley     3
```

```

Maserati      2
ISUZU         2
Ferrari       1
Mercedes-AMG  1
Rolls-Royce   1
Force         1
Name: count, dtype: int64

===== model =====
model
i20          906
Swift Dzire   890
Swift         781
Alto          778
City          757
...
Altroz        1
C              1
Ghost         1
Quattroporte  1
Gurkha        1
Name: count, Length: 120, dtype: int64

===== seller_type =====
seller_type
Dealer        9539
Individual     5699
Trustmark Dealer  173

```

9. Understand the Target Variable

Our goal is to predict `selling_price`.

This is the target variable, also called the dependent variable.

Before training the model, we need to understand how the target values are distributed.

We will inspect:

- minimum selling price
- maximum selling price
- average selling price
- median selling price
- distribution of prices

Understanding the target helps us identify unusual values and understand the problem we are trying to predict.

```
df["selling_price"].describe()
```

```

count    1.541100e+04
mean     7.749711e+05
std      8.941284e+05
min      4.000000e+04
25%      3.850000e+05
50%      5.560000e+05
75%      8.250000e+05
max      3.950000e+07
Name: selling_price, dtype: float64

```

One important ML concept

We now have:

Features / X ————— brand model vehicle_age km_driven seller_type fuel_type transmission_type mileage
engine max_power seats

and:

Target / y ————— selling_price

Our ML model will learn a relationship roughly like:

X —————→ y car information price

For example:

Hyundai + i20 + 5 years + 20,000 km + Petrol + Manual ↓ ML model ↓ ₹5–6 lakh

The actual prediction will be learned from the dataset rather than us manually creating the rule.

10. Visualize the Target Variable

A numerical summary gives us useful statistics, but a graph can show the distribution more clearly.

We will plot the distribution of `selling_price` to see:

- where most car prices are concentrated
- whether the distribution is symmetric
- whether there are very expensive outliers

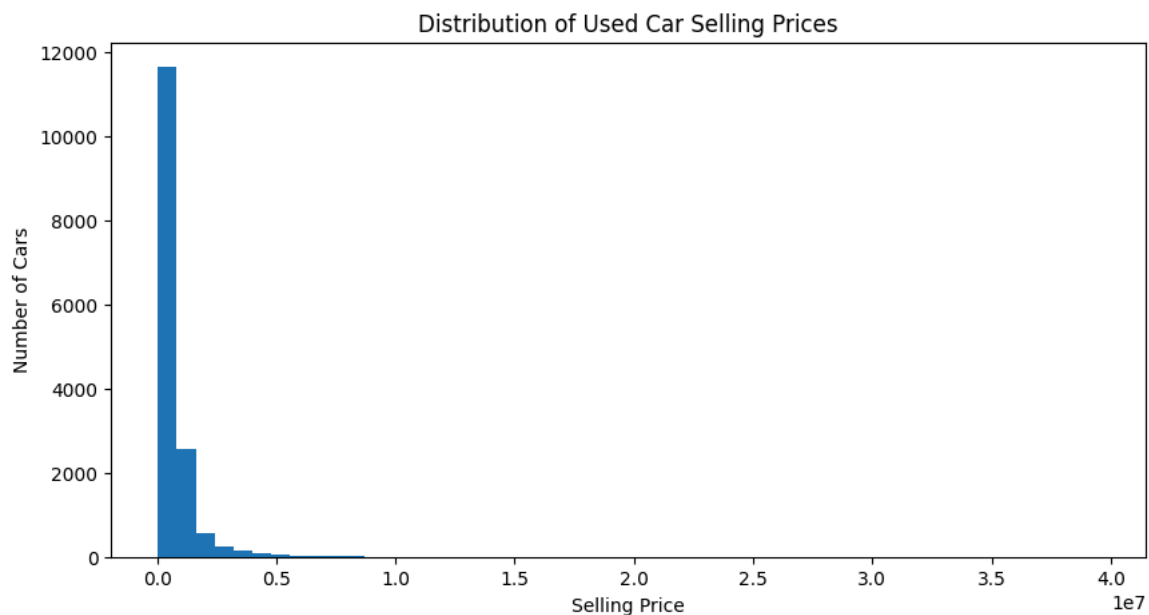
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.hist(df["selling_price"], bins=50)

plt.xlabel("Selling Price")
plt.ylabel("Number of Cars")
plt.title("Distribution of Used Car Selling Prices")

plt.show()
```



11. Investigate High-Priced Cars

The target distribution is strongly right-skewed.

Before removing or transforming any values, we will inspect the cars with the highest selling prices.

This helps us determine whether the extreme prices are genuine observations or possible data errors.

```
df.sort_values("selling_price", ascending=False)[
    ["car_name", "brand", "model", "selling_price"]
].head(10)
```

	car_name	brand	model	selling_price
3799	Ferrari GTC4Lusso	Ferrari	GTC4Lusso	39500000
10969	Rolls-Royce Ghost	Rolls-Royce	Ghost	24200000
1172	Bentley Continental	Bentley	Continental	14500000
9722	Mercedes-Benz S-Class	Mercedes-Benz	S-Class	13000000
9364	Porsche Cayenne	Porsche	Cayenne	11100000
10989	Mercedes-Benz S-Class	Mercedes-Benz	S-Class	11000000
1888	Mercedes-Benz S-Class	Mercedes-Benz	S-Class	11000000
11000	Land Rover Rover	Land Rover	Rover	9200000
8439	BMW 7	BMW	7	8500000
3096	BMW 7	BMW	7	8500000

12. Understand Price Percentiles

The mean and maximum price can be strongly affected by a small number of expensive cars.

Percentiles help us understand the typical price range of the dataset without being dominated by extreme values.

We will inspect several percentiles of the selling price.

```
df["selling_price"].quantile(0.50)
```

```
np.float64(556000.0)
```

```
df["selling_price"].quantile(
    [0.25, 0.50, 0.75, 0.90, 0.95, 0.99]
)
```

```
0.25    385000.0
0.50    556000.0
0.75    825000.0
0.90   1375000.0
0.95   2050000.0
0.99   4545000.0
Name: selling_price, dtype: float64
```

13. Separate Features and Target

Our goal is to predict the `selling_price` of a used car.

Therefore:

- `selling_price` is our target variable (y).
- The remaining useful columns are our input features (X).

The model will learn a relationship between the car's characteristics and its selling price.

In simple terms:

Features → Model → Predicted Selling Price

```
X = df.drop(
    columns=["selling_price", "Unnamed: 0", "car_name"]
)
```

```
y = df["selling_price"]
```

```
print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (15411, 11)
Target shape: (15411,)
```

14. Inspect the Model Features

Before encoding categorical variables, we will separate the features into:

1. Numerical features
2. Categorical features

This is important because different types of data require different preprocessing.

Numerical values can generally be used directly by many machine-learning models.

Categorical values such as `brand`, `fuel_type`, and `transmission_type` must be converted into a numerical representation.

```
print("Numerical columns:")
print(X.select_dtypes(include=["int64", "float64"]).columns)
```

```
print("\nCategorical columns:")
print(X.select_dtypes(include=["object"]).columns)
```

```
Numerical columns:
Index(['vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats'], dtype='object')
```

```
Categorical columns:
Index(['brand', 'model', 'seller_type', 'fuel_type', 'transmission_type'], dtype='object')
```

15. Before One-Hot Encoding

Our categorical features contain different numbers of categories:

- `brand` → 32 categories
- `model` → 120 categories
- `seller_type` → 3 categories
- `fuel_type` → 5 categories
- `transmission_type` → 2 categories

Machine-learning models generally require numerical input.

We therefore need to convert these categorical values into numerical representations.

For this project, we will use One-Hot Encoding for the categorical features.

One-Hot Encoding creates a separate binary feature for each category.

For example, if `fuel_type` contains:

Petrol, Diesel, CNG, LPG, Electric

it can become:

- `fuel_type_Petrol`
- `fuel_type_Diesel`
- `fuel_type_CNG`
- `fuel_type_LPG`
- `fuel_type_Electric`

A Petrol car would have:

Petrol = 1

and the other fuel categories would be:

Diesel = 0 CNG = 0 LPG = 0 Electric = 0

This allows the machine-learning model to work with categorical information without assuming that one category is numerically greater than another.

```
for column in X.select_dtypes(include=["object"]).columns:  
    print(column, "→", X[column].nunique(), "unique values")
```

```
brand → 32 unique values  
model → 120 unique values  
seller_type → 3 unique values  
fuel_type → 5 unique values  
transmission_type → 2 unique values
```

```
categorical_columns = X.select_dtypes(include=["object"]).columns.tolist()  
numerical_columns = X.select_dtypes(  
    include=["int64", "float64"]  
) .columns.tolist()
```

```
print("Categorical columns:")  
print(categorical_columns)
```

```
print("\nNumerical columns:")  
print(numerical_columns)
```

```
Categorical columns:  
['brand', 'model', 'seller_type', 'fuel_type', 'transmission_type']
```

```
Numerical columns:  
['vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power', 'seats']
```

✓ 16. Prepare the Encoding and Preprocessing

Our dataset contains both numerical and categorical features.

Numerical features can be used directly by many machine-learning models:

- `vehicle_age`
- `km_driven`
- `mileage`
- `engine`

- `max_power`
- `seats`

Categorical features contain text values:

- `brand`
- `model`
- `seller_type`
- `fuel_type`
- `transmission_type`

Machine-learning models need numerical representations, so we will use One-Hot Encoding for the categorical features.

We will use `ColumnTransformer` to apply the encoder only to categorical columns while leaving numerical columns unchanged.

Later, we will place this preprocessing inside a `Pipeline` together with our machine-learning model.

This gives us a clean sequence:

Data → Preprocessing → Model → Prediction

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ],
    remainder="passthrough"
)

print("Preprocessor created successfully.")

Preprocessor created successfully.
```

✓ 17. Understand Fit and Transform

Before using the preprocessor, we need to understand two important operations:

- `fit()` learns the required information from the data.
- `transform()` applies the learned information to data.

For One-Hot Encoding, `fit()` learns which categories exist in each categorical column.

For example, if `fuel_type` contains:

- Petrol
- Diesel
- CNG
- LPG
- Electric

the encoder learns these categories during `fit()`.

Then `transform()` uses those learned categories to convert values into numerical columns.

`fit_transform()` performs both operations together.

However, during machine-learning training we must be careful about where `fit()` is performed. We should learn preprocessing information from the training data and then use that learned preprocessing on the test data.

This prevents information from the test set from leaking into the training process.

```
example = X[["fuel_type"]].head(5)

print("Original data:")
print(example)
```

```
Original data:
fuel_type
0    Petrol
1    Petrol
2    Petrol
3    Petrol
4    Diesel
```

17.1 What Does `fit()` Learn?

The encoder needs to learn which categories exist before it can convert them.

We will use only the `fuel_type` column for this small demonstration.

The `fit()` operation learns the categories from the data, but does not yet return the encoded dataset.

After fitting, we can inspect the categories learned by the encoder.

```
fuel_encoder = OneHotEncoder()

fuel_encoder.fit(example)

print("Categories learned by the encoder:")
print(fuel_encoder.categories_)
```

```
Categories learned by the encoder:
[array(['Diesel', 'Petrol'], dtype=object)]
```

17.2 Transform Categories into Numbers

The `fit()` operation taught the encoder which categories exist.

Now we will use `transform()` to convert the categories into numerical values.

The encoder has learned:

- Diesel
- Petrol

It will create a separate numerical column for each category.

This is the actual one-hot encoding step.

```
encoded_example = fuel_encoder.transform(example)

print("Encoded data:")
print(encoded_example.toarray())
```

```
Encoded data:
[[0.  1.]
 [0.  1.]
 [0.  1.]
 [0.  1.]
 [1.  0.]]
```

18. Split Data into Training and Testing

Before training the machine-learning model, we divide our dataset into two parts:

- Training data → used to teach the model.
- Testing data → used to evaluate the model on unseen data.

We will use 80% of the data for training and 20% for testing.

The target variable is `selling_price`.

The features are stored in `X` and the target is stored in `y`.

Important:

The preprocessing steps must be learned from the training data only.

This helps prevent data leakage from the test set.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
```

```
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
```

```
Training features: (12328, 11)
Testing features: (3083, 11)
Training target: (12328,)
Testing target: (3083,)
```

19. Fit the Preprocessor on Training Data

We have now separated the dataset into training and testing data.

The preprocessor should learn categorical information from the training data only.

Therefore:

- `fit_transform(X_train)` learns the categories from the training data and converts the training features.
- `transform(X_test)` uses the same learned categories to convert the testing features.

We do not fit the preprocessor separately on the test data.

This prevents information from the test set from leaking into the training process.

```
X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

print("Processed training shape:", X_train_processed.shape)
print("Processed testing shape:", X_test_processed.shape)
```

```
Processed training shape: (12328, 165)
Processed testing shape: (3083, 165)
```

20. Create the Random Forest Model

We have now completed the preprocessing stage.

The processed training data contains:

- 12,328 training records
- 165 numerical features

The processed testing data contains:

- 3,083 testing records
- 165 numerical features

Our selected machine-learning algorithm is `RandomForestRegressor`.

Random Forest is an ensemble regression algorithm. It builds many decision trees and combines their predictions to produce a final numerical prediction.

In this project, the model will learn the relationship between the car's features and its `selling_price`.

The important distinction is:

- `preprocessor` prepares the data.
- `RandomForestRegressor` learns patterns from the prepared data.
- `predict()` will later use those learned patterns to estimate selling prices.

We will start with 100 decision trees.

`random_state=42` makes the model's random behavior reproducible.

`n_jobs=-1` allows the model to use all available CPU cores where parallel processing is supported.

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

print("Random Forest model created successfully.")

Random Forest model created successfully.
```

21. Train the Random Forest Model

The Random Forest model has now been created, but it has not learned anything yet.

Training happens when we call `fit()`.

We provide:

- `X_train_processed` → the processed training features
- `y_train` → the correct selling prices for those training records

The model uses these examples to learn patterns between the car's characteristics and its selling price.

The model is trained only on the training data.

We do not use `X_test_processed` or `y_test` for training.

The workflow is:

Training data:

`X_train_processed + y_train → model.fit() → trained model`

Testing will happen later:

`X_test_processed → model.predict() → predicted prices`

This separation is important because the test data should represent unseen data when we evaluate the model.

```
model.fit(
    X_train_processed,
    y_train
)

print("Random Forest training completed.")
```

22. Make Predictions on the Test Data

The Random Forest model has now been trained using the training data.

We can use the trained model to predict selling prices for the test dataset.

The test features were not used during model training.

This gives us an unbiased set of predictions that we can later compare with the actual selling prices.

The workflow is:

`X_test_processed → trained Random Forest → predicted selling prices`

The resulting predictions will contain one predicted selling price for each test car.

```
y_pred = model.predict(X_test_processed)

print("Number of predictions:", len(y_pred))
print("First 10 predictions:")
print(y_pred[:10])
```

23. Compare Actual and Predicted Prices

The model has generated predictions for all 3,083 test cars.

We now compare the predicted selling prices with the actual selling prices.

This helps us understand how closely the Random Forest model is predicting the real prices.

We will first inspect a small sample of actual and predicted values before calculating evaluation metrics.

```
comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

print(comparison.head(10))
```

24. Evaluate the Random Forest Model

The model has generated predictions for all test records.

We will now evaluate the predictions using three regression metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

MAE tells us the average absolute prediction error.

RMSE gives more importance to large prediction errors.

R^2 measures how well the model explains the variation in selling prices.

These metrics will help us objectively evaluate the performance of the Random Forest model on unseen test data.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print("Model Evaluation")
print("-----")
print("MAE :", mae)
print("RMSE:", rmse)
print("R2 :", r2)
```

25. Visualize Actual vs Predicted Prices

We have already calculated the model evaluation metrics.

Now we will visualize the relationship between the actual selling prices and the prices predicted by the Random Forest model.

If the predictions are close to the actual values, the points should generally follow a diagonal pattern.

A perfect model would place every point exactly on the diagonal line.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred, alpha=0.4)

plt.xlabel("Actual Selling Price")
plt.ylabel("Predicted Selling Price")
plt.title("Actual vs Predicted Selling Prices")

# Perfect prediction reference line
min_price = min(y_test.min(), y_pred.min())
max_price = max(y_test.max(), y_pred.max())

plt.plot(
    [min_price, max_price],
    [min_price, max_price],
    linestyle="--"
)

plt.show()
```

26. Save the Trained Model and Preprocessor

Now that the Random Forest model has been trained and evaluated, we need to save it for later use.

The trained model will be used in our Django application to predict the selling price of used cars.

We also save the preprocessor because new data from the Django form must go through the same preprocessing steps that were used during model training.

The workflow in our Django application will be:

User Input → Preprocessor → Trained Random Forest Model → Predicted Car Price

We will use the `joblib` library to save both the trained model and the preprocessor as `.joblib` files.

```
import joblib

joblib.dump(model, "/kaggle/working/car_price_model.joblib")

joblib.dump(preprocessor, "/kaggle/working/preprocessor.joblib")

print("Model saved successfully.")
print("Preprocessor saved successfully.")
```

27. Verify the Saved Model Files

After saving the trained model and preprocessor, we should verify that both files exist successfully.

These files will later be downloaded from Kaggle and added to our Django project.

The two important files are:

- `car_price_model.joblib` — contains the trained Random Forest model.
- `preprocessor.joblib` — contains the preprocessing pipeline used to prepare input data.

It is important to keep both files because the Django application must preprocess new user input in the same way as the training data before making a prediction.

```
import os

model_path = "/kaggle/working/car_price_model.joblib"
preprocessor_path = "/kaggle/working/preprocessor.joblib"

print("Model exists:", os.path.exists(model_path))
print("Preprocessor exists:", os.path.exists(preprocessor_path))

print("\nSaved files:")

print(
    "car_price_model.joblib size:",
    os.path.getsize(model_path),
    "bytes"
)

print(
    "preprocessor.joblib size:",
    os.path.getsize(preprocessor_path),
    "bytes"
)
```

28. Test the Saved Model and Preprocessor

Before using the saved model in our Django application, we should verify that the model and preprocessor can be loaded successfully.

We will load both `.joblib` files from disk and use them to make a prediction.

This simulates what will happen later in the Django application:

User Input → Preprocessor → Loaded Model → Predicted Car Price

If the loaded model can successfully make predictions, it confirms that our saved machine learning files are ready for integration.

```
import joblib

# Load the saved model
loaded_model = joblib.load("/kaggle/working/car_price_model.joblib")

# Load the saved preprocessor
loaded_preprocessor = joblib.load("/kaggle/working/preprocessor.joblib")

print("Model loaded successfully.")
print("Preprocessor loaded successfully.")

print("\nModel type:")
print(type(loaded_model))

print("\nPreprocessor type:")
print(type(loaded_preprocessor))
```

29. Test Prediction Using the Loaded Model

We have successfully loaded the trained Random Forest model and the preprocessing pipeline from their saved `.joblib` files.

Now we will test whether the loaded files can make predictions correctly.

We will use a small sample from the test dataset and process it using the loaded preprocessor.

The processed data will then be passed to the loaded Random Forest model.

This simulates the workflow that will later be used in our Django application:

New Car Data → Loaded Preprocessor → Loaded Model → Predicted Selling Price

If the loaded model successfully produces predictions, it confirms that both saved files are working correctly and are ready to be integrated into our Django application.

```
# Take the first 5 records from the original test data
sample_data = X_test.head(5)

# Transform the data using the loaded preprocessor
sample_processed = loaded_preprocessor.transform(sample_data)

# Make predictions using the loaded model
sample_predictions = loaded_model.predict(sample_processed)

# Display the predictions
for i, prediction in enumerate(sample_predictions, start=1):
    print(f"Car {i} predicted price: ₹ {prediction:,.2f}")
```

30. Create a Reusable Prediction Function

To prepare our machine learning model for Django integration, we will create a reusable prediction function.

The function will:

1. Accept new car data
2. Apply the saved preprocessor
3. Use the trained model
4. Return the predicted selling price

This structure will later make it easier to connect our machine learning model with a Django form and web interface.

```
import pandas as pd

def predict_car_price(car_data):

    # Convert input data into a DataFrame
    input_df = pd.DataFrame([car_data])

    # Apply preprocessing
    processed_data = loaded_preprocessor.transform(input_df)

    # Make prediction
    prediction = loaded_model.predict(processed_data)

    # Return predicted price
    return prediction[0]
```

31. Test the Reusable Prediction Function

We will now test the reusable prediction function using one car record from the test dataset.

This confirms that the function can:

1. Accept car information as input
2. Convert the information into a DataFrame
3. Apply the saved preprocessor
4. Generate a prediction using the trained model
5. Return the predicted selling price

This is very similar to how our Django application will later send user-entered car information to the machine learning model.

```
# Take one car record from the test dataset
sample_car = X_test.iloc[0].to_dict()

# Predict the selling price
predicted_price = predict_car_price(sample_car)
```

```
print("Predicted selling price: ₹", round(predicted_price, 2))  
print("Actual selling price: ₹", round(y_test.iloc[0], 2))
```